Univerzitet u Novom Sadu
Fakultet tehničkih nauka
Primenjeno softversko inženjerstvo

ELEMENTI RAZVOJA SOFTVERA

Vežba 1

Zahtevi, backlog, Git i timski razvoj

Radni materijal za računarske vežbe, samostalno ponavljanje i projektni rad.

Akademska 2026/2027. godina
Dopunjena Vežba 1: korisničke priče i simulirani sprintovi u PYXIS AI.

Sadržaj

Vežba 1 - Zahtevi, backlog, Git i timski razvoj 3

1.1. User Story i kriterijumi prihvatanja 3

1.2. Backlog, Tapiz Boards i plan rada 4

1.3. Git kao sledljiv zapis razvoja 14

1.4. Pull request, pregled koda i konflikti 15

Vežba 1 - Zahtevi, backlog, Git i timski razvoj

Razvoj softvera počinje razumevanjem problema, ali se kvalitet tog razumevanja potvrđuje tek kada zahtev postane proverljiva promena u proizvodu. U ovoj vežbi zahtevi, planiranje rada i Git ne posmatraju se kao odvojene teme. Oni čine jedan razvojni tok: poslovna potreba se oblikuje kao stavka u backlog-u, razrađuje kroz kriterijume prihvatanja, implementira na izdvojenoj grani, pregleda kroz pull request i integriše tek nakon provere.

Od zahteva do integrisane promene

01	**Potreba** problem ili korisnička vrednost
02	**Backlog** prioritet, opis i kriterijumi
03	**Grana** izolovana implementacija
04	**Pull request** pregled, testovi i diskusija
05	**Integracija** proverena promena u glavnoj grani

Slika 1.1 - Razvojni trag treba da omogući timu da poveže zahtev, odluku, implementaciju i proveru rezultata.

> **Osnovna ideja**
> Git istorija nije rezervna kopija završnog rešenja. Backlog nije spisak želja. Pull request nije formalnost. Svaki od ovih elemenata predstavlja deo proverljivog procesa razvoja.

1.1. User Story i kriterijumi prihvatanja

User Story predstavlja sažet zapis potrebe korisnika ili drugog učesnika u poslovnom procesu. Njegova svrha nije da unapred propiše klasu, tabelu ili tehnološko rešenje, već da jasno odredi kome je funkcionalnost potrebna, koji rezultat se očekuje i zbog čega taj rezultat ima vrednost.

> **Primer**
> Kao nastavnik laboratorije, želim da rezervišem slobodnu opremu za određeni termin, kako bih unapred znao da je oprema dostupna i sprečio dvostruku rezervaciju.

Kriterijumi prihvatanja pretvaraju opšti zahtev u skup uslova koji mogu da se provere testom ili demonstracionim scenarijem. Dobro napisan kriterijum opisuje spolja vidljivo ponašanje sistema i izbegava nepotrebno vezivanje za detalje implementacije.

Vrsta scenarija	Primer kriterijuma
Uspešan tok	Rezervacija se kreira kada je oprema dostupna, korisnik ovlašćen, a vremenski period ispravan.

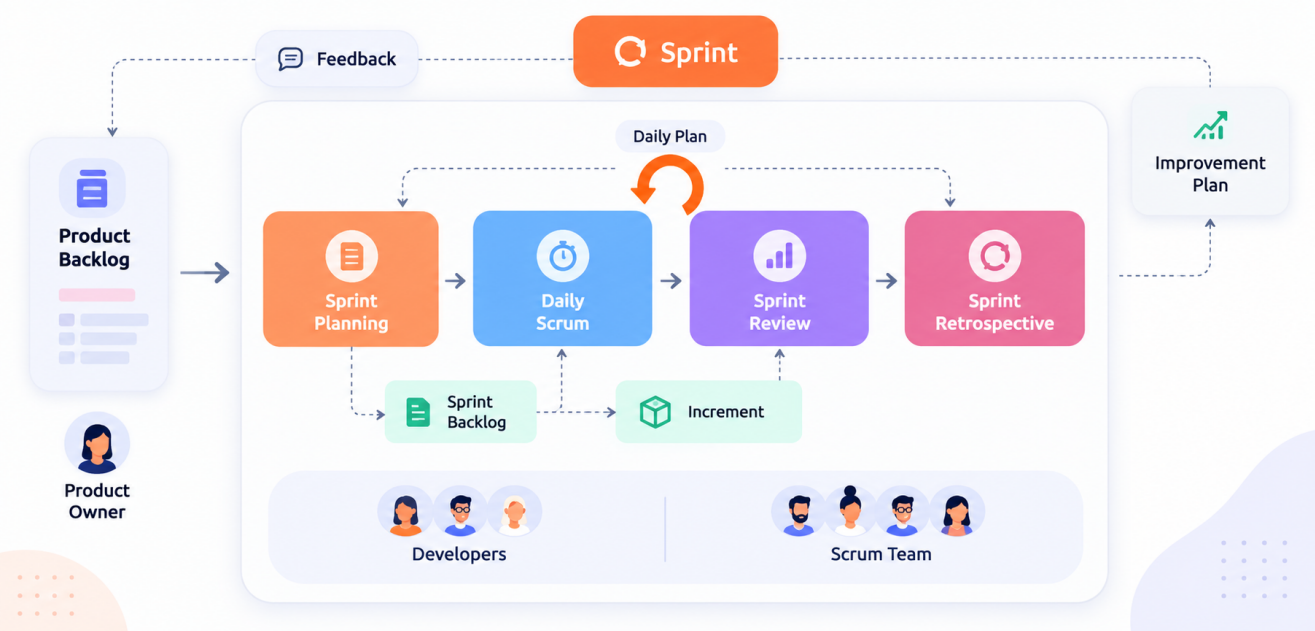
Vrsta scenarija	Primer kriterijuma
Poslovni konflikt	Sistem odbija zahtev kada se period preklapa sa postojećom aktivnom rezervacijom iste opreme.
Neispravan ulaz	Sistem odbija zahtev kada početak termina nije pre njegovog kraja.
Granični slučaj	Pravilo mora jasno da odredi da li nova rezervacija može početi tačno u trenutku završetka prethodne.

Tabela 1.1.1

> **Rad na vežbi**
> Za jednu stavku projektnog domena napisati User Story, najmanje dva uspešna i tri negativna ili granična kriterijuma prihvatanja. Za svaki kriterijum navesti kako će se proveriti.

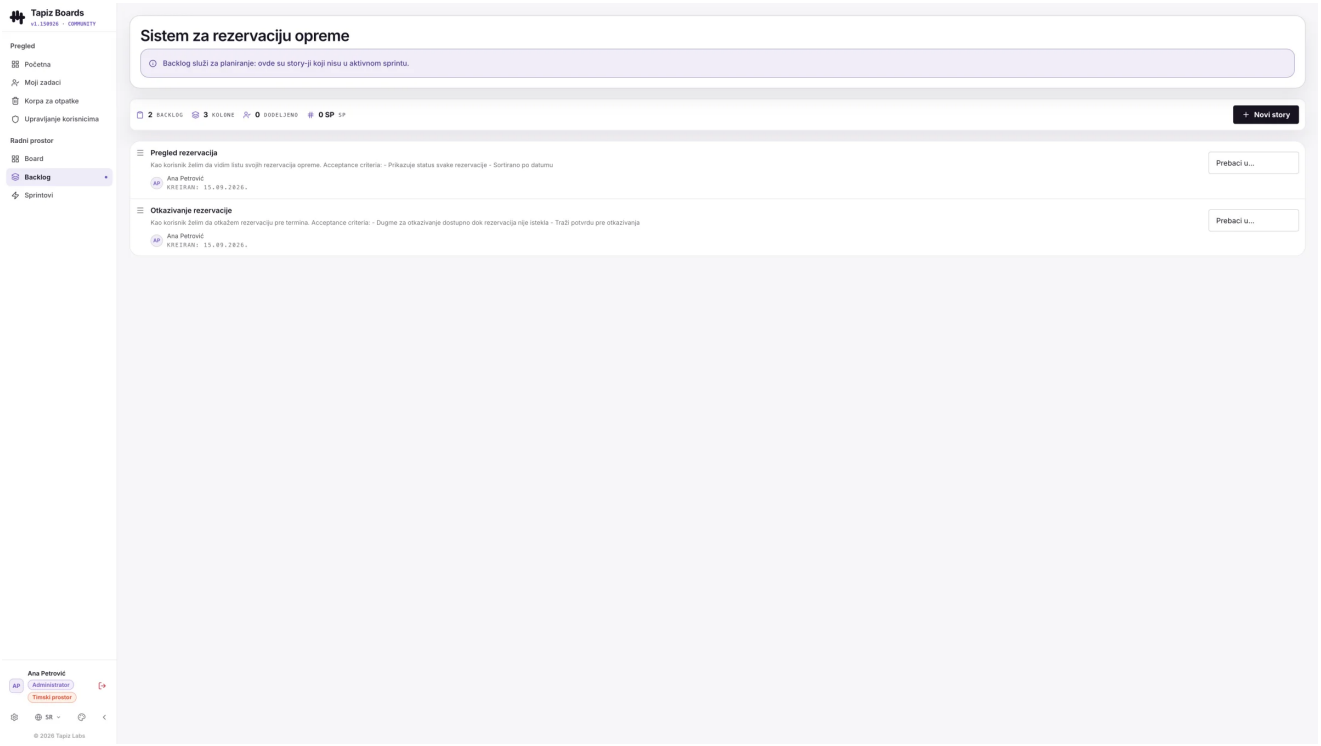
1.2. Backlog, Tapiz Boards i plan rada

Rad tima se odvija u ponovljivim ciklusima (sprintovima) unutar kojih se planira, implementira i preispituje ograničen obim posla. Product Backlog je zajednički izvor svih budućih stavki; Sprint Backlog i Increment nastaju kada tim za tekući sprint izabere i realizuje deo tog posla.



Slika 1.2.1 - Product Backlog napaja Sprint Planning; tim potom prolazi kroz Daily Scrum, Sprint Review i Sprint Retrospective, dok povratne informacije i plan unapređenja zatvaraju ciklus.

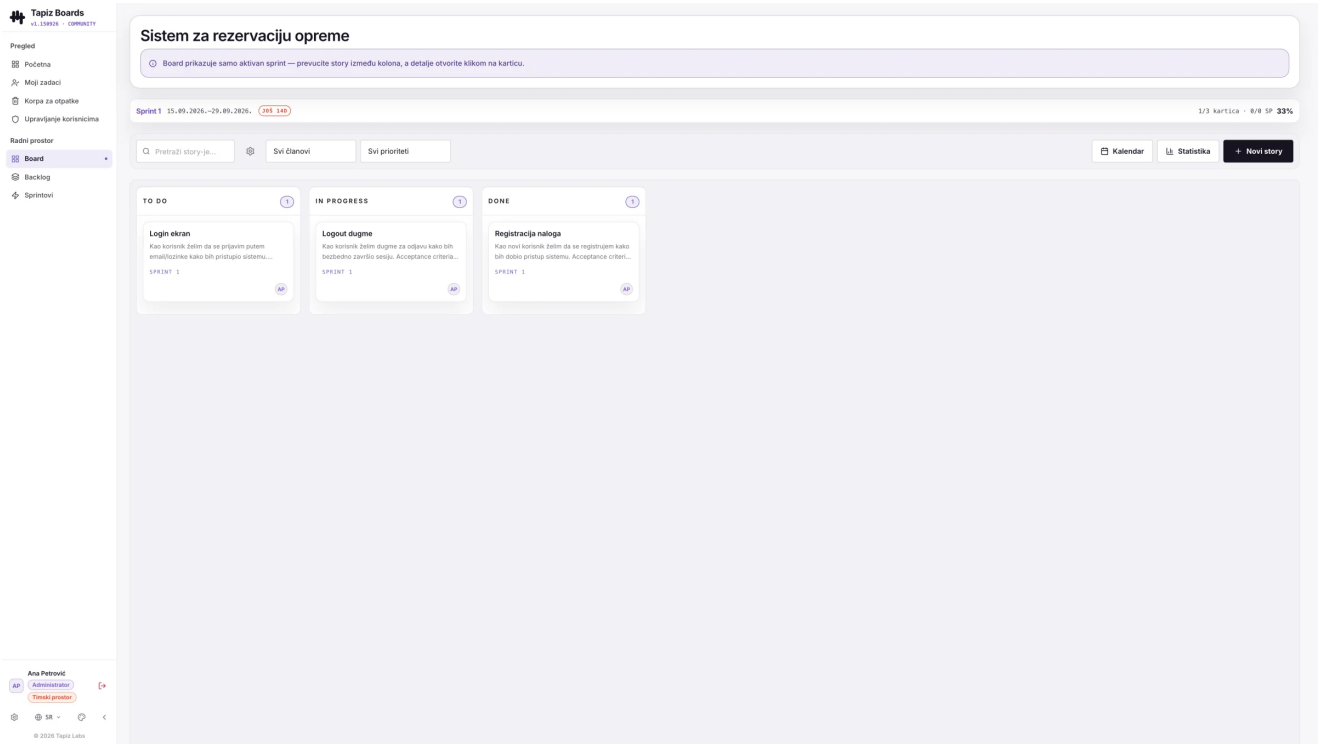
Backlog je uređena lista rada, a ne arhiva nepovezanih ideja. Stavka koja ulazi u aktivan rad treba da ima dovoljno jasan cilj, prioritet, kriterijume prihvatanja i poznata ograničenja da tim može da proceni njen obim i započne implementaciju bez nagađanja o osnovnom poslovnom smislu.



Slika 1.2.2 - Product Backlog u Tapiz Boards-u omogućava timu da sagleda prioritete, stanje pripreme i plan narednog rada.

Status	Značenje
Backlog	Stavka postoji, ali još nije spremna za neposredan rad.
Ready	Cilj, kriterijumi i osnovna ograničenja su dovoljno jasni za početak rada.
In Progress	Implementacija je u toku i postoji odgovorna osoba ili par.
Code Review	Promena je implementirana i čeka stručni pregled koda.
QA/Verify	Proverava se ponašanje sistema u odnosu na kriterijume prihvatanja.
Done	Promena je integrisana i ispunjava dogovorenu Definition of Done.

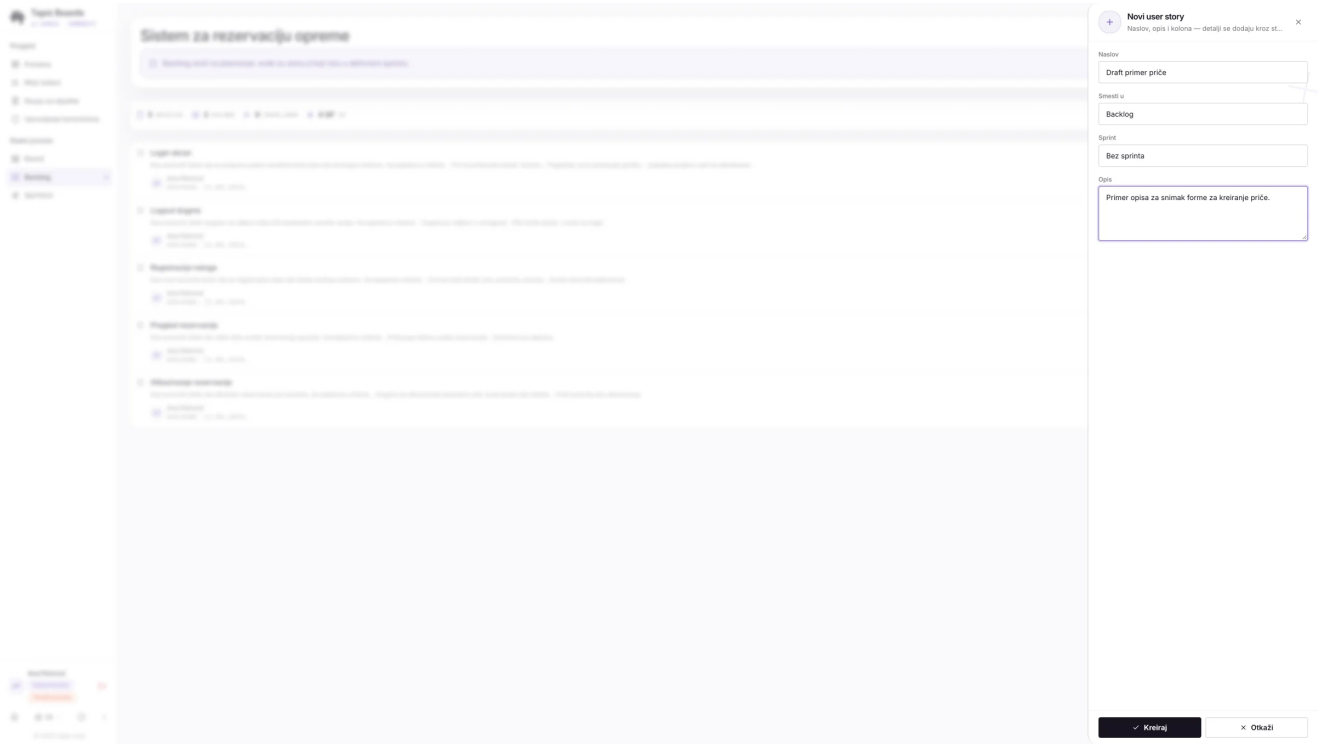
Tabela 1.2.1



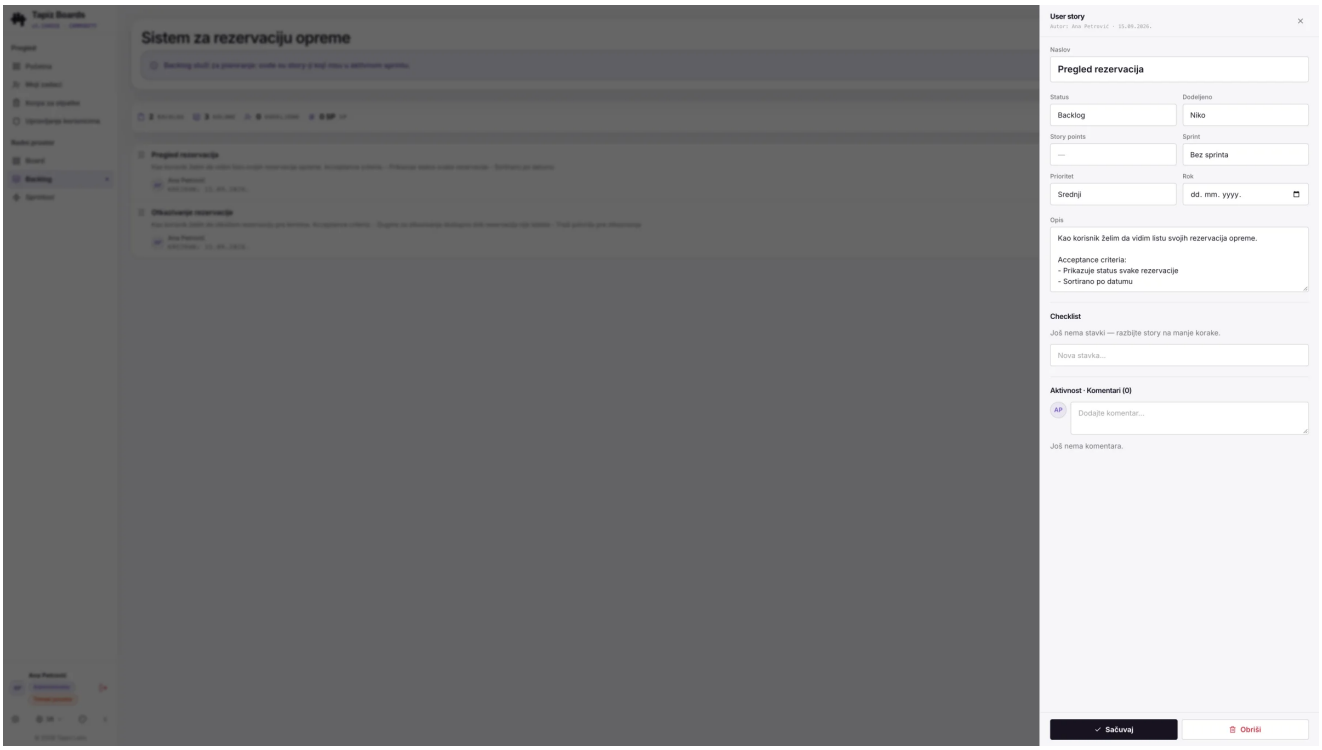
Slika 1.2.3 - Radna tabla prikazuje stvarno stanje stavki u dogovorenom razvojnom procesu.

> **Status mora odgovarati stvarnom stanju**
> Kartica se ne smatra završenom zato što je ručno premeštena u poslednju kolonu. Status Done znači da su ispunjeni kriterijumi prihvatanja, izvršene potrebne provere i integrisana odgovarajuća promena u repozitorijumu.

Svaka stavka backlog-a je više od naslova. Detalj priče objedinjuje opis, kriterijume prihvatanja, prioritet, dodeljenu osobu, procenu obima i vezu sa sprintom - sve na jednom mestu, tako da se odluka o tome da li je stavka spremna za rad ne oslanja na usmeni dogovor ili razbacane napomene.

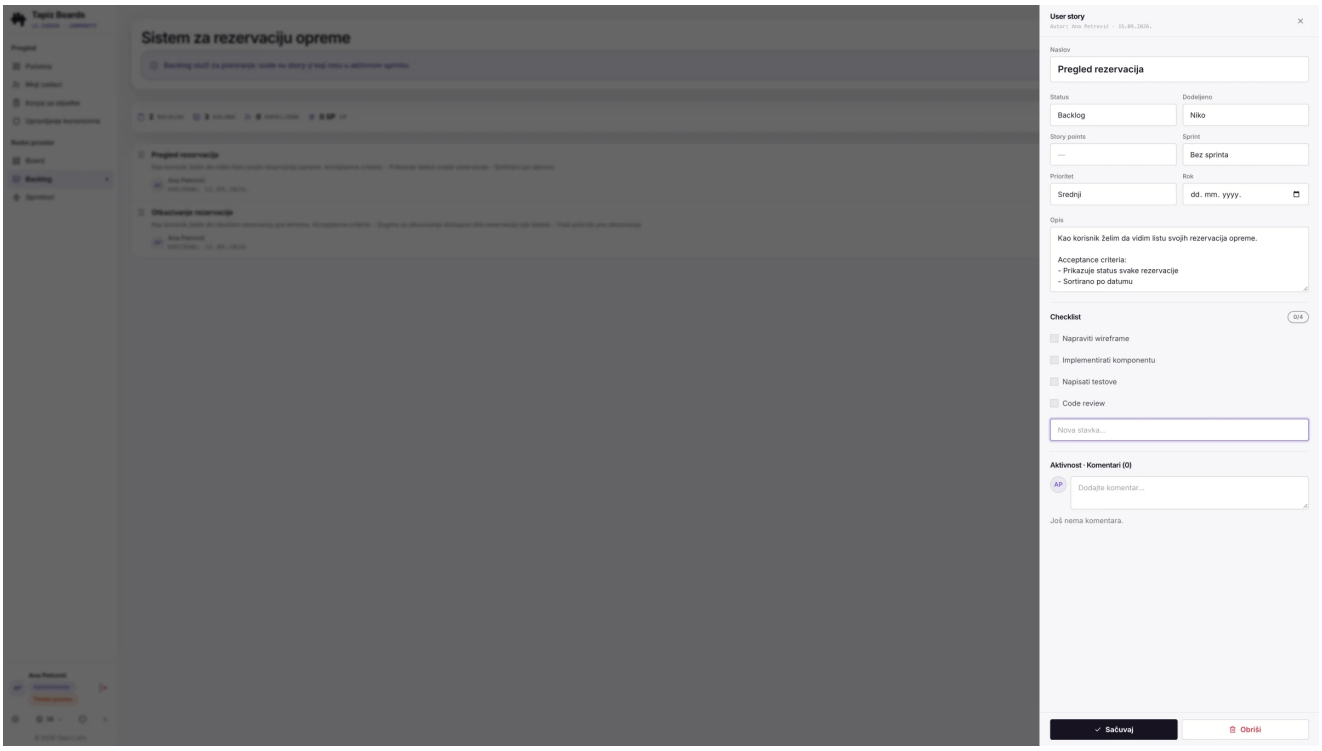


Slika 1.2.4 - Forma za kreiranje ili izmenu stavke: naslov, kolona, sprint i opis se definišu pre nego što stavka uđe u aktivan rad.



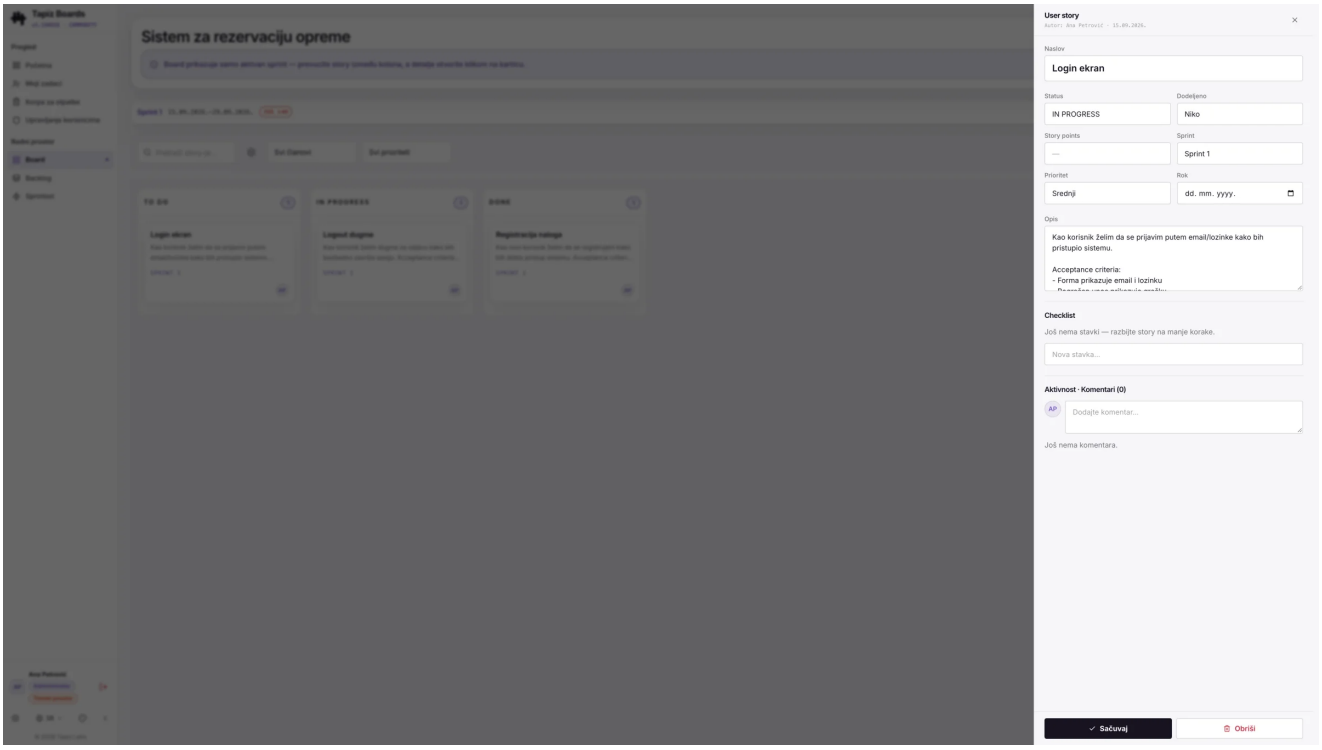
Slika 1.2.5 - Kriterijumi prihvatanja se pišu kao deo opisa priče, u obliku koji tim može direktno da prevede u testove ili demonstracioni scenario.

Kada je stavka dovoljno velika ili nejasna da bi se implementirala u jednom koraku, razbija se na manje, proverljive korake kroz checklistu. Ovo nije birokratija - to je način da se veliki, rizičan zadatak pretvori u niz malih, nezavisno proverljivih koraka.



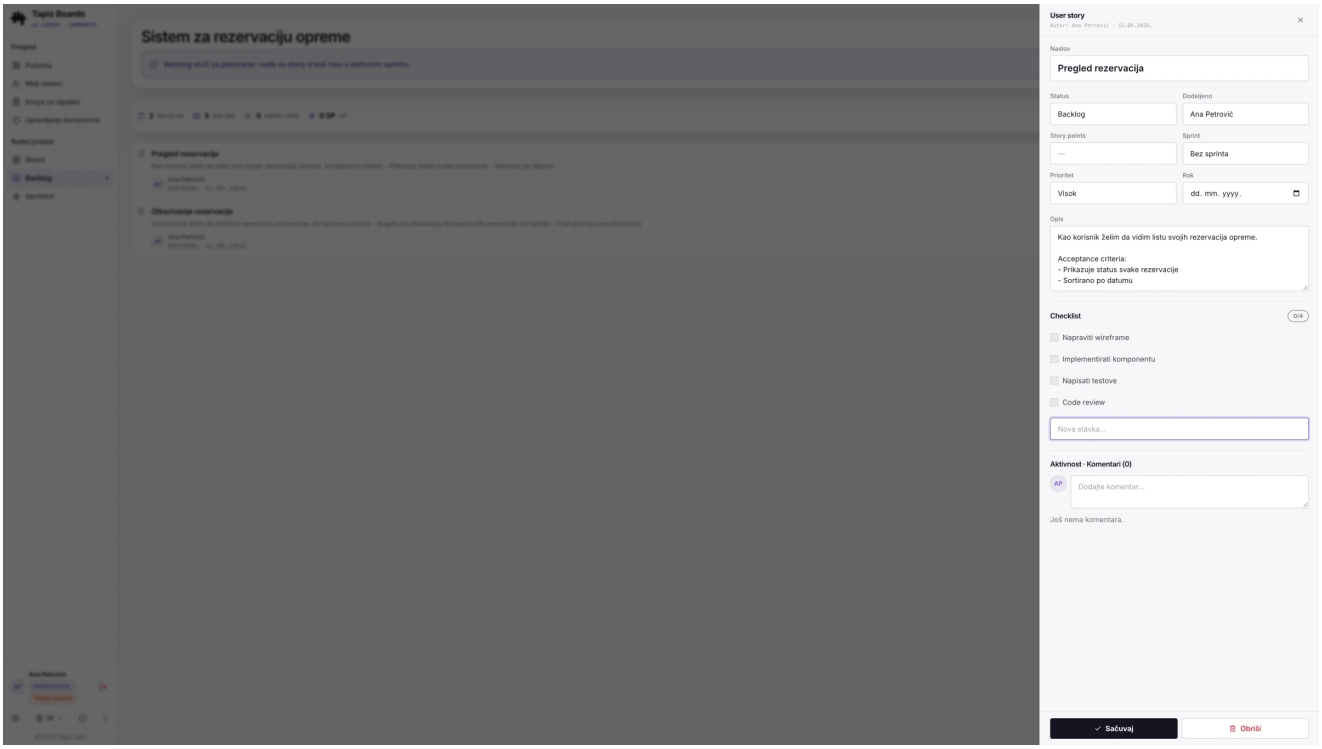
Slika 1.2.6 - Checklist unutar priče razlaže rad na male korake (wireframe, implementacija, testovi, code review) koje tim može pojedinačno da prati i otkačinje.

Board prikazuje samo stavke aktivnog sprinta i njihov status u dogovorenom toku. Premeštanje kartice između kolona treba da prati stvarnu promenu stanja rada, a ne da bude kozmetička radnja pred sastanak.

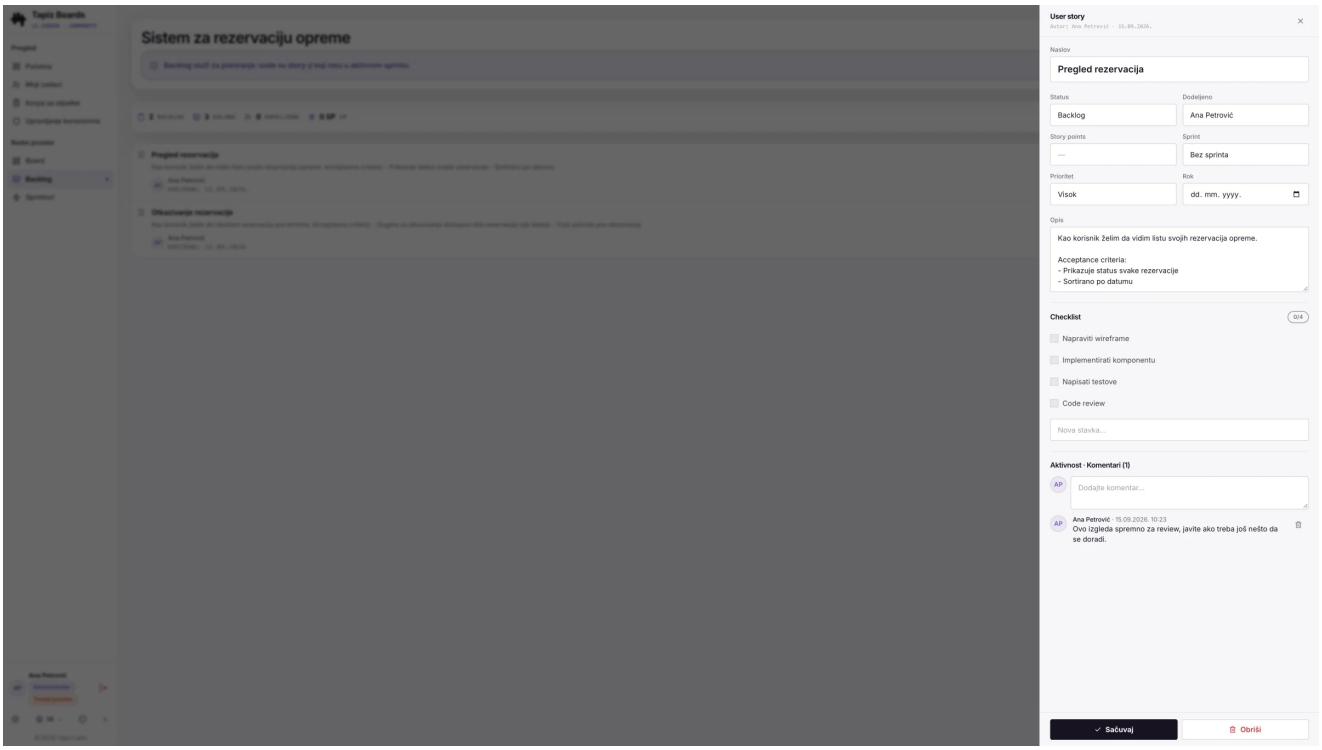


Slika 1.2.7 - Otvorena priča u statusu In Progress, sa vidljivim sprintom i vezom prema koloni na tabli - status u detalju i status na tabli su isti podatak, prikazan na dva mesta.

Prioritet i dodeljena osoba pomažu timu da odgovori na pitanje šta raditi sledeće i ko je odgovoran za ishod, dok se aktivnost i komentari čuvaju kao trag odluka i dogovora vezanih za tu stavku - umesto da se izgube u razgovoru van alata.



Slika 1.2.8 - Visok prioritet i dodeljena osoba su vidljivi na samoj stavci, zajedno sa checklistom koja pokazuje koliko je posla još preostalo.



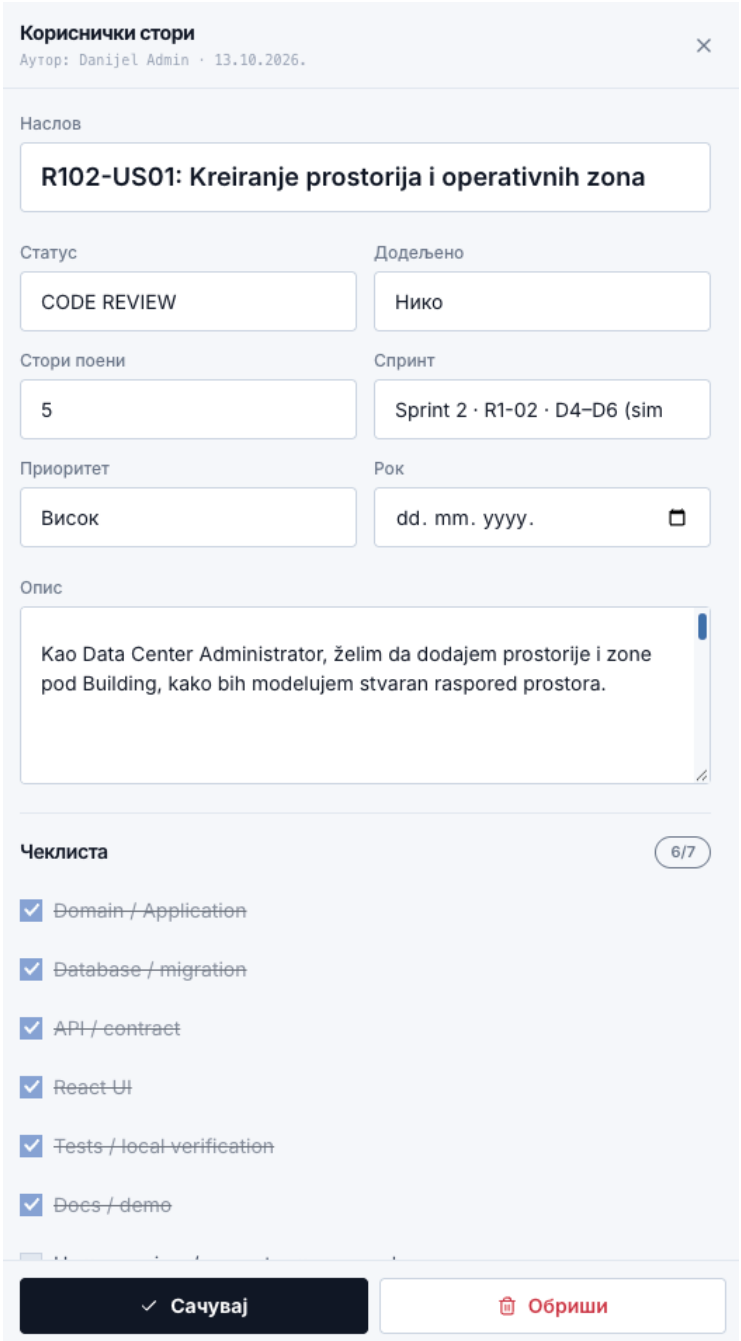
Slika 1.2.9 - Komentar na stavci beleži dogovor tima o narednom koraku, čineći odluku sledljivom i nezavisnom od usmenog dogovora.

> **Tapiz Boards je alat, ne cilj**
> Svrha ovih ekrana nije da se nauči jedan konkretan alat, već da se vidi kako backlog, kriterijumi prihvatanja, raspodela na zadatke, status i trag odluka rade zajedno kao jedan koherentan proces planiranja rada.

1.2.1. Primer korisničke priče i sprinta - PYXIS AI

Sledeći ekrani prikazuju konkretan primer rada sa korisničkim pričama u Tapiz Boards-u, na projektu „PYXIS AI · R1 · simulirani sprintovi“. Na njima se vidi kako se opis potrebe povezuje sa procenom obima, checklistom, sprintom i statusom rada.

Detalj korisničke priče



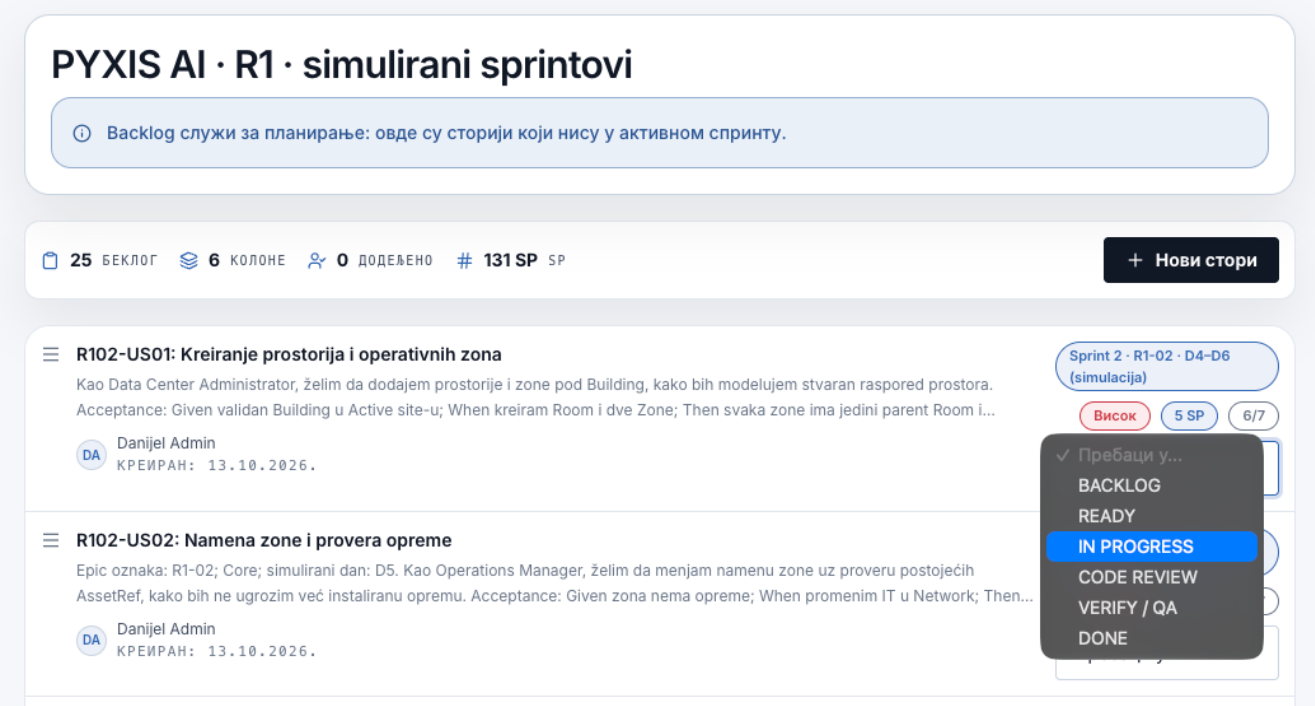
Slika 1.2.1.1 - Detalj priče R102-US01: opis potrebe, plan rada i napredak checkliste.

Priča „R102-US01: Kreiranje prostorija i operativnih zona“ sadrži naslov, status, dodeljenu osobu, stori poene, sprint, prioritet, rok, opis i checklistu. Opis prati obrazac „Kao [uloga], želim [funkcionalnost], kako bih [korist]“: administrator data centra želi da doda prostorije i zone pod objektom Building kako bi modelovao stvaran raspored prostora.

Procena od 5 SP (Story Points) predstavlja relativnu procenu obima i složenosti rada. Visok prioritet govori o važnosti priče, dok polje „Dodeljeno“ određuje odgovornost za rad. Na ovom snimku priča je u statusu CODE REVIEW, a polje „Dodeljeno“ ima vrednost „Niko“ - tim treba da dogovori odgovornu osobu.

Checklist prikazuje korake kao što su Domain / Application, Database / migration, API / contract, React UI, Tests / local verification i Docs / demo. Oznaka 6/7 znači da je označeno šest od sedam koraka. Kriterijumi prihvatanja zasebno određuju očekivano ponašanje sistema; završeni tehnički koraci sami po sebi ne potvrđuju da su ti kriterijumi ispunjeni.

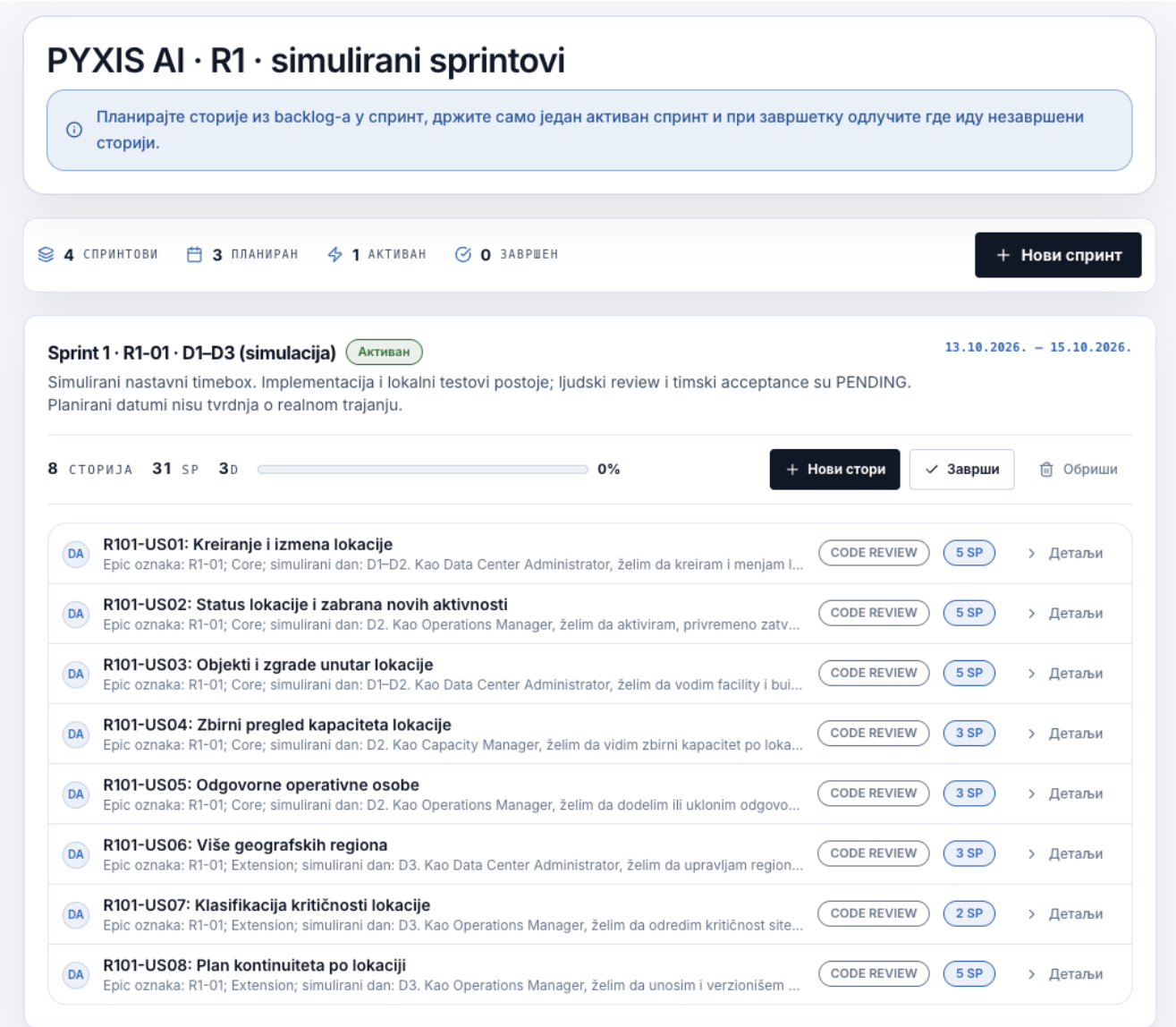
Lista priča i promena statusa



Slika 1.2.1.2 - Lista korisničkih priča i izbor statusa rada.

U listi su vidljivi naslov, sažetak opisa i kriterijuma prihvatanja, autor, sprint, prioritet, procena u SP i napredak checkliste. Otvoreni meni prikazuje dogovorene statuse: BACKLOG, READY, IN PROGRESS, CODE REVIEW, VERIFY / QA i DONE. Tim bira status koji odgovara stvarnom stanju rada. Izbor statusa opisuje napredak priče, dok izbor sprinta određuje u kom ciklusu se ona planira.

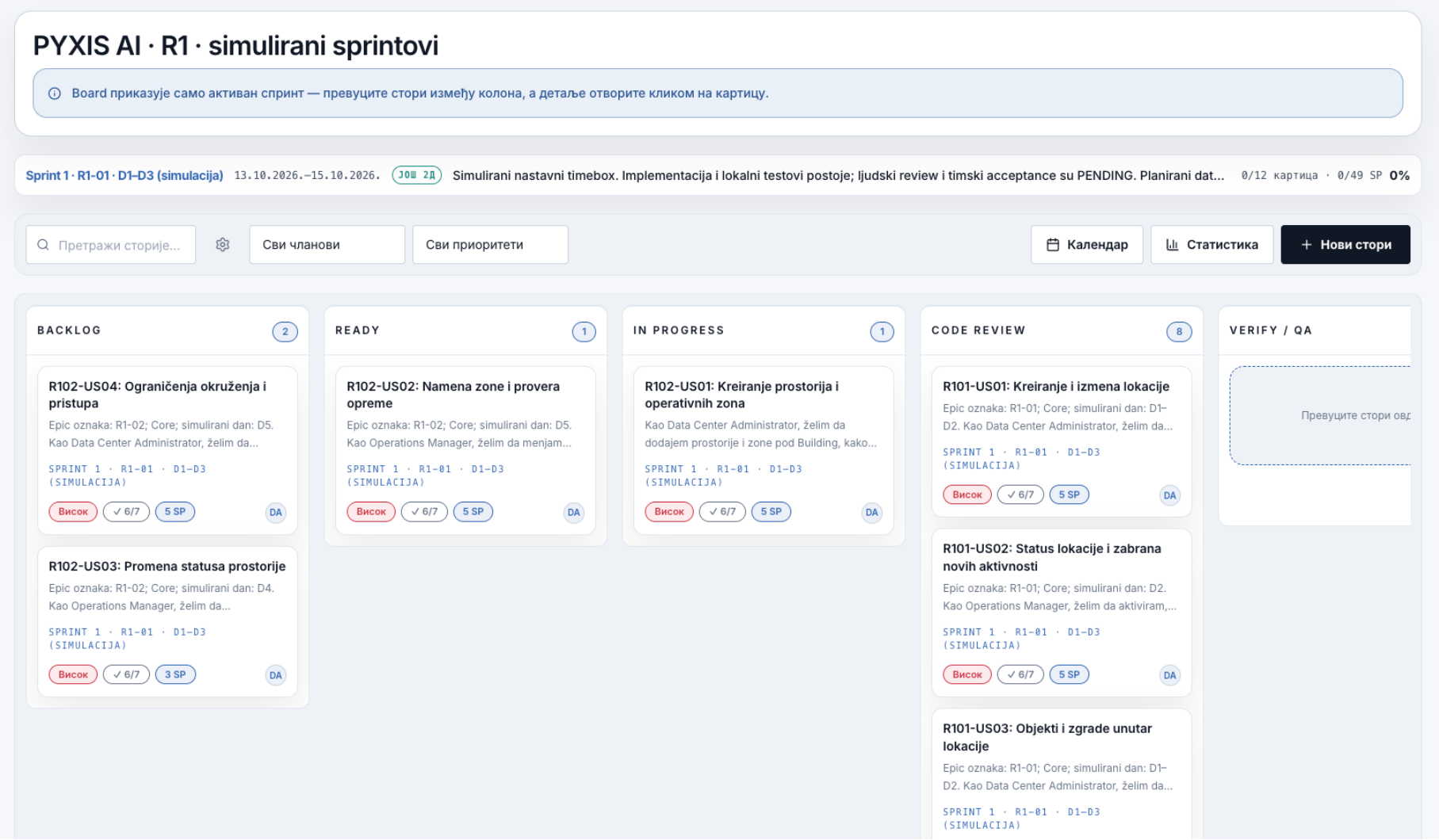
Sprint i njegove korisničke priče



Slika 1.2.1.3 - Aktivni simulirani sprint: osam priča, 31 SP i pregled statusa.

Prikazan je aktivni „Sprint 1 · R1-01 · D1-D3 (simulacija)“, sa osam priča i ukupnom procenom od 31 SP. Pored svake priče nalaze se njen status, procena i pristup detaljima. Sve prikazane priče su u statusu CODE REVIEW, dok je napredak sprinta 0%: implementacija i lokalni testovi mogu da postoje, a da stručni pregled i timsko prihvatanje još nisu završeni. Oznake D1-D3 i datumi služe nastavnoj simulaciji.

Radna tabla aktivnog sprinta



Slika 1.2.1.4 - Radna tabla aktivnog sprinta sa karticama raspoređenim po statusima.

Radna tabla grupiše kartice po statusima BACKLOG, READY, IN PROGRESS, CODE REVIEW i VERIFY / QA; desni deo table nije u celosti obuhvaćen snimkom. Kartica objedinjuje naslov, kratak opis, oznaku sprinta, prioritet, checklistu, procenu i avatar autora. Detalji se otvaraju klikom na karticu, a promena statusa može se obaviti prevlačenjem između kolona. Veći broj kartica u CODE REVIEW ukazuje timu na posao koji čeka pregled.

Snimci predstavljaju različite trenutke nastavne simulacije, pa se broj priča, njihov status i pripadnost sprintu mogu razlikovati između prikaza. Pri poređenju ekrana istu priču prepoznajemo po oznaci, na primer R102-US01.

> **Rad na vežbi - od priče do table**
> Za jednu priču svog projekta popuniti naslov, opis, kriterijume prihvatanja, prioritet, odgovornu osobu, procenu i checklistu. Planirati je u sprintu, otvoriti njenu karticu na tabli i obrazložiti koji dokaz je potreban za prelazak iz IN PROGRESS u CODE REVIEW, zatim u VERIFY / QA i DONE.

1.3. Git kao sledljiv zapis razvoja

Git omogućava da se razvoj rekonstruiše kroz male i smislene promene. Radno stablo sadrži trenutne izmene, staging area određuje šta ulazi u naredni commit, a commit predstavlja imenovanu tačku istorije. U timskom radu posebna grana izdvaja jednu promenu od stabilne glavne grane.

Pojam	Uloga u radu tima
Working tree	Datoteke koje se trenutno menjaju.
Staging area	Izbor sadržaja koji će ući u naredni commit.
Commit	Koherentna promena sa porukom koja opisuje njenu nameru.
Branch	Izdvojena linija razvoja za jednu funkcionalnost, ispravku ili drugi ograničen posao.
Remote	Udaljeni repozitorijum preko kog tim razmenjuje promene.

Tabela 1.3.1

Listing 1.3.1 - BASH

Primer malog i proverljivog razvojnog koraka

```
git status
git diff
git add src/Application/ReservationService.cs
git add tests/ReservationServiceTests.cs
git diff --staged
git commit -m "Add reservation overlap rule"
git push -u origin feature/reservation-overlap
```

- Pre commit-a pregledati pripremljeni diff i razumeti svaku uključenu promenu.
- Jedan commit treba da predstavlja jednu koherentnu nameru kad god je to praktično moguće.
- Promene koje nisu povezane sa ciljem stavke ne treba usput uključivati u isti pull request.
- Glavna grana treba da ostane stabilna i spremna za zajednički rad tima.

1.4. Pull request, pregled koda i konflikti

Pull request je mesto na kome se zahtev povezuje sa konkretnom implementacijom. Pregledalac proverava da li je promena u odgovarajućem obimu, da li poštuje arhitektonska pravila, da li postoje potrebni testovi i da li se iz diff-a može razumeti namera autora. Tek nakon pregleda i provere promena se integriše u glavnu granu.

Tok jedne promene

01	**Stavka** jasan cilj i kriterijumi
02	**Grana** ograničena implementacija
03	**Commit-i** smisleni razvojni koraci
04	**Pregled** kod, testovi i rizici
05	**QA i merge** provereno ponašanje i integracija

Slika 1.4.1 - Pull request povezuje poslovni zahtev sa implementacijom i dokazima provere.

Konflikt pri spajanju nastaje kada Git ne može samostalno da odredi konačni sadržaj. Rešavanje konflikta nije uklanjanje tehničkih markera, već odluka o tome koje ponašanje treba da ostane nakon spajanja paralelnih promena.

Listing 1.4.1 - BASH

Rešavanje konflikta uz završnu proveru

```bash
git checkout main
git pull
git checkout feature/reservation-overlap
git merge main
# rešiti konflikt i pregledati konačan diff
git add src/Application/ReservationService.cs
git commit -m "Resolve reservation rule conflict"
dotnet test
```

Prepisivanje zajedničke istorije
`git push --force` nije deo uobičajenog timskog toka. Na deljenim granama može ukloniti tuđe commit-e i koristi se samo kada tim razume posledice i postoji opravdan razlog.

Ishod vežbe
Student ume da poveže zahtev, backlog, Git istoriju, pull request i proveru ponašanja u jedan sledljiv razvojni proces.